

Relatório de Laboratório 02

Assistentes de IA vs. Codificação Manual: um experimento controlado

Campo	Informação
Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Laboratório 02 — Assistentes de IA vs. Codificação Manual
Grupo (trio)	Artur Candiar · João Vitor Santana · Pedro Maia
Repositório / GitHub Projects	https://github.com/candian15/LabExpGRP
Data de entrega	24/09/2026

1. Introdução

Ferramentas de IA generativa tornaram-se onipresentes no desenvolvimento de software, mas ainda há pouca evidência controlada e reproduzível sobre seu real impacto em produtividade e qualidade — a maior parte do que se afirma é relato anedótico. Este laboratório realiza um experimento controlado do tipo **crossover within-subject, time-boxed**, para avaliar quantitativamente os efeitos do uso de um assistente de IA na resolução de tarefas de programação (katas), comparando cada integrante consigo mesmo com e sem IA.

As Questões de Pesquisa do enunciado são:

- **RQ1** — O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?
- **RQ2** — O uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?
- **RQ3** — O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

Hipóteses informais do grupo, antes de olhar os dados: esperávamos redução expressiva de tempo com IA (H1-alt), taxa de sucesso igual ou levemente superior com IA (H2-alt), e pouca ou nenhuma diferença estrutural relevante em katas curtos (H3-nula provável).

Inovações do grupo (30%): (a) intervalo de confiança da mediana por reamostragem bootstrap, para quantificar a incerteza sob N pequeno; (b) normalização das métricas estruturais por LOC como controle explícito de verbosidade; (c) gráfico pareado (slopegraph) coerente com o desenho within-subject. Detalhadas na seção 3.6.

2. Contexto

Este é o Lab02 da disciplina, sequência do Lab01 (mineração de repositórios). O objeto de estudo é o próprio processo de resolução de katas de programação, com e sem assistente de IA, sob tempo controlado. Foram usados seis katas autorais candidatos, dos quais quatro compuseram o experimento final — todos de dificuldade média e baixa indexação, criados pelo grupo justamente para reduzir o risco de o assistente reproduzir soluções memorizadas de exercícios clássicos.

A base conceitual é o método GQM (Goal-Question-Metric) de Basili, Caldiera & Rombach: o Goal define comparar IA vs. codificação manual quanto a tempo, defeitos e estrutura; as Questions são as RQ1–RQ3; e as Metrics são as descritas na seção 3.5.

3. Metodologia

3.1 Principais Desafios

- Padronizar katas de dificuldade equivalente e evitar memorização pela IA: resolvido criando katas autorais com regras próprias do grupo (moedas arbitrárias, formato de log inventado, regras de senha não convencionais).
- Coletar tempo de forma objetiva e reproduzível: resolvido com um cronômetro automatizado que roda os testes em laço e para exatamente no "time-to-green".
- Padronizar os dados dos três integrantes num formato único, após uma primeira coleta com colunas divergentes entre integrantes.
- Tamanho amostral pequeno (12 trials): tratado com estatística robusta (mediana/IQR, Wilcoxon e bootstrap) em vez de média/desvio-padrão.

3.2 Tomadas de Decisão

- **Assistente de IA:** Claude (versão gratuita, via chat em claude.ai), o mesmo em todos os trials de IA. Autocomplete de IA na IDE foi desativado, para que o único canal de IA fosse o chat e o tratamento fosse comparável.
- **Linguagem:** Python, escolhida em função da ferramenta de métricas estáticas (Radon), já que CK exige Java.
- **Time-box:** 35 minutos por trial, encerrado ao final independentemente do resultado; trials incompletos seriam censurados em 35 min (não ocorreu nenhum).
- **Limite de WIP:** coluna Doing limitada a 3 itens, um por integrante, para refletir o trabalho paralelo do trio sem acúmulo.
- **Contrabalanceamento:** cada kata foi resolvido por uns integrantes com IA e por outros manualmente, em ordens opostas, para controlar variação individual de habilidade.

3.3 Etapas

O trabalho foi organizado em três sprints mais o relatório final, com Issues individuais rastreadas no GitHub Projects (board "Trabalho Medição"). A divisão abaixo reflete os Assignees reais das Issues.

Sprint	Entregas	Responsável(is)
S01 — Desenho + Preparação	Scripts de coleta (timer, testes, métricas), katas autorais e desenho experimental	Pedro (infra/scripts e testes dos katas); trio (desenho)
S02 — Execução	12 trials (4 katas × 3 integrantes), metade com IA, metade manual	João, Pedro e Artur (cada um seus 4 trials)
S03 — Análise + Dashboard	Wilcoxon RQ1/RQ2, análise RQ3 com bootstrap, dashboard	Artur (RQ1/RQ2); Pedro (RQ3); João (dashboard)
Relatório Final	Documento consolidado e discussão	Trio

Configuração do processo — board com as colunas **Backlog** → **To Do** → **Doing** → **Review** → **Done**, WIP de Doing = 3. A captura do board ao final do laboratório deve ser inserida abaixo.

[Inserir aqui o print do quadro Kanban do GitHub Projects — arquivo enviado pelo grupo.]

3.4 Ferramentas

- Python 3.12 — scripts próprios de coleta (timer.py, run_tests.py, metrics.py, run_trial.py).
- pytest — execução dos testes de aceitação dos katas.
- Radon — complexidade ciclomática (cc), Índice de Manutenibilidade (mi) e LOC. jscpd — duplicação.
- Pandas — manipulação dos dados; Matplotlib/Seaborn — visualização (dashboard).
- SciPy — teste de Wilcoxon para amostras pareadas; bootstrap implementado pelo grupo.

- GitHub Projects (v2) — gestão do processo. Repositório: <https://github.com/candian15/LabExpGRP>

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta
RQ1	Time-to-green	Tempo do início do trial até todos os testes passarem (ou 35 min se censurado)	segundos	timer.py
RQ2	Taxa de sucesso	testes passando ÷ testes totais × 100, ao fim do time-box	%	run_tests.py / pytest
RQ3	Complexidade ciclomática	Média de McCabe por função, normalizada por LOC	nº / 100 LOC	Radon cc
RQ3	Duplicação	% de linhas duplicadas	%	jscpd
RQ3	LOC (controle)	Linhas de código da solução	linhas	Radon
RQ3	Manutenibilidade	Índice de Manutenibilidade (complexidade + LOC + Halstead)	0–100	Radon mi

3.6 Inovações Propostas pelo Grupo (30% da nota)

(a) Intervalo de confiança da mediana por bootstrap. Dado o N pequeno (12 trials), a fórmula paramétrica de IC não é confiável. Reamostramos os valores com reposição 10.000 vezes e tomamos os percentis 2,5% e 97,5% das medianas. Isso quantifica a incerteza e aparece na discussão da RQ3, onde os intervalos de IA e manual se sobrepõem — mostrando que diferenças estruturais observadas podem não ser reais.

(b) Normalização por LOC. Código de IA pode ser mais verboso; comparar complexidade ou duplicação sem normalizar engana. Reportamos as métricas estruturais também por 100 LOC, como controle explícito exigido metodologicamente.

(c) Slopegraph pareado. Em vez do gráfico de barras usual, ligamos manual→IA por linhas, tornando visível o efeito within-subject kata a kata (Figura 2).

4. Resultados

4.1 Coleta de Dados

Foram concluídos 12 trials (4 katas × 3 integrantes), todos dentro do time-box de 35 minutos — nenhum censurado. Cada integrante resolveu 2 katas com IA e 2 manualmente, em ordem contrabalanceada. Não houve dados ausentes; nenhum outlier foi removido, pois a distribuição é coerente com o esperado. A duplicação foi 0% em todos os trials, esperado dado o tamanho reduzido dos katas.

4.2 Visualização Gráfica

RQ1 — O uso de IA reduz o tempo? A mediana com IA foi de 3,7 min contra 12,6 min manual — redução de ~70%. A redução é consistente entre os três integrantes (68–70%).

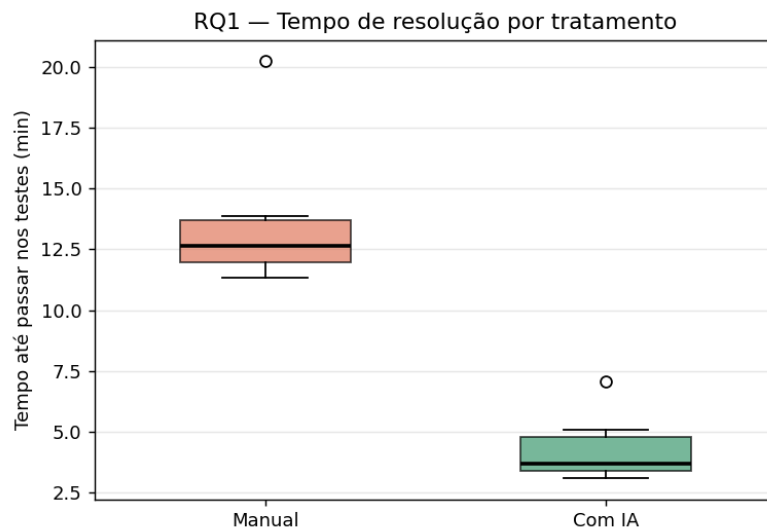


Figura 1 — Tempo de resolução por tratamento (boxplot). Mediana manual $\approx 12,6$ min; com IA $\approx 3,7$ min.

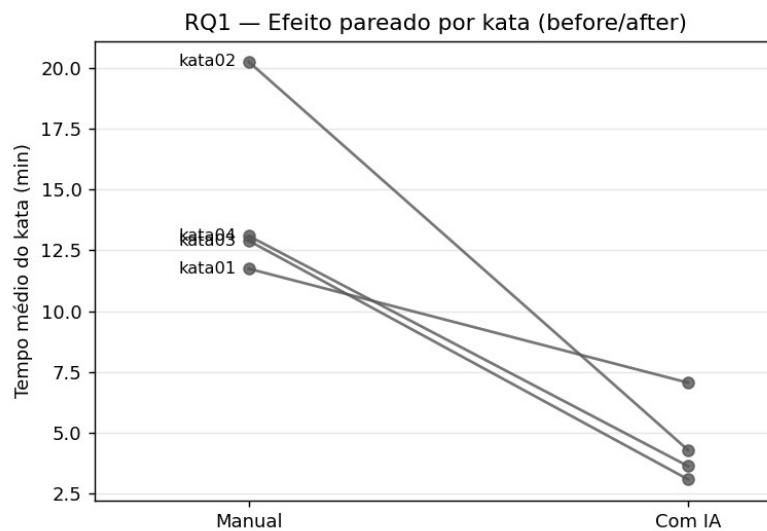


Figura 2 — Efeito pareado por kata (inovação): todas as linhas descem de manual para IA.

RQ2 — O uso de IA reduz defeitos? A taxa de sucesso foi 100% em ambos os tratamentos: todos os 12 trials passaram em todos os testes dentro do tempo. Não houve diferença de defeitos — os katas foram resolvíveis nos dois modos dentro do time-box.

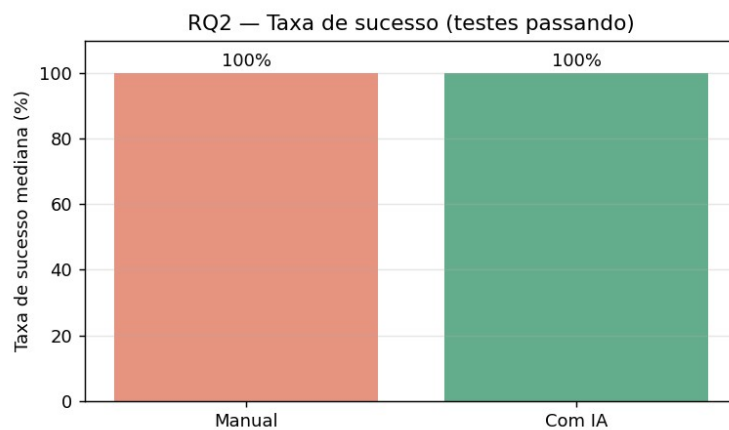


Figura 3 — Taxa de sucesso mediana por tratamento (ambas 100%).

RQ3 — O uso de IA altera complexidade/duplicação? As medianas de complexidade (5,5) e duplicação (0%) foram idênticas entre os tratamentos. O Índice de Manutenibilidade foi levemente maior com IA (90,1 vs

88,0), mas os intervalos de confiança se sobrepõem. O LOC mediano foi igual (24,5), indicando que a IA não gerou código mais verboso nestes katas.

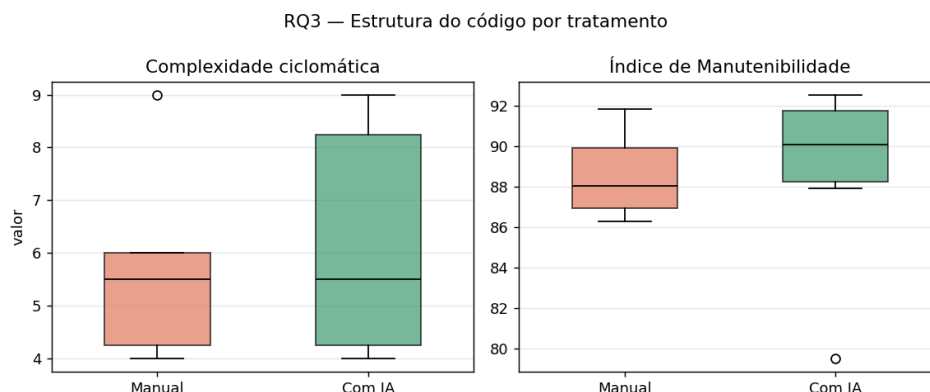


Figura 4 — Complexidade ciclomática e Índice de Manutenibilidade por tratamento.

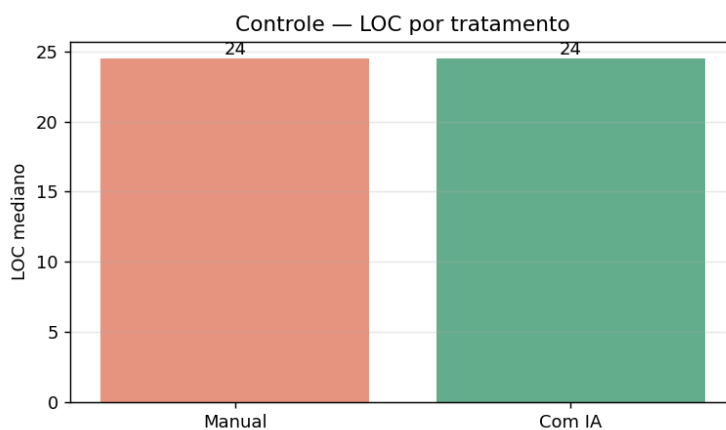


Figura 5 — LOC como métrica de controle (mediana igual entre tratamentos).

4.3 Discussão

RQ1 (confirmada). A hipótese de redução expressiva de tempo com IA foi confirmada: ~70% de redução na mediana, consistente entre integrantes. O teste de Wilcoxon pareado por kata resultou em $p = 0,125$ — não atinge significância a 5%, o que é esperado com apenas 4 pares (o menor p possível com 4 pares é 0,125). O tamanho e a consistência do efeito, porém, são fortes indícios práticos a favor da IA, ainda que a amostra limite a força estatística formal.

RQ2 (nula não rejeitada). Não houve diferença de defeitos: 100% de sucesso nos dois tratamentos. Em katas curtos e bem especificados, ambos os modos alcançam a solução correta dentro do tempo — a IA acelera, mas não muda a taxa de acerto neste contexto.

RQ3 (nula não rejeitada). Complexidade e duplicação idênticas; manutenibilidade levemente superior com IA, mas com intervalos de bootstrap sobrepostos — não é possível afirmar diferença real. A normalização por LOC confirma que a IA não produziu código mais verboso aqui.

Ameaças à validade. Efeito de aprendizado entre katas (mitigado pelo contrabalanceamento); familiaridade prévia com a ferramenta de IA; e o risco de memorização — reduzido pelo uso de katas autorais pouco indexados. O N pequeno é a principal limitação de validade estatística, explicitamente tratada pelo bootstrap.

Contribuição das inovações. O bootstrap e a normalização por LOC não contradisseram os 70% do enunciado; eles qualificaram os achados, mostrando que a vantagem clara da IA está no tempo (RQ1), enquanto as diferenças estruturais (RQ3) não se sustentam sob a incerteza amostral.

5. Conclusão

O experimento indica que, no contexto de katas curtos e autorais resolvidos por estudantes sob tempo controlado, o assistente de IA reduz fortemente o tempo de resolução (~70% na mediana), sem alterar a taxa de acerto (100% em ambos) nem a estrutura do código de forma estatisticamente sustentável. Em síntese: a IA entregou velocidade sem custo aparente de qualidade neste cenário.

As principais limitações são o tamanho amostral (12 trials, 4 pares por RQ), que limita a força do teste de Wilcoxon, e o escopo restrito a katas curtos — os resultados não se estendem automaticamente a tarefas grandes e ambíguas, onde o efeito da IA sobre defeitos e manutenibilidade pode ser diferente.

Com mais tempo, o grupo expandiria a amostra (mais katas e mais integrantes ou repetições) para dar poder estatístico ao Wilcoxon, registraria o número de prompts por trial para uma análise qualitativa do esforço de interação, e incluiria katas de maior porte para testar se a vantagem de tempo se mantém quando a complexidade cresce. Entre as inovações, o bootstrap da mediana é a que mais valeria a pena expandir, por endereçar diretamente a fragilidade de amostras pequenas típica deste tipo de estudo.

6. Referências

- BASILI, V.; CALDIERA, G.; ROMBACH, H. D. The Goal Question Metric Approach. Encyclopedia of Software Engineering, 1994.
- ZUSE, Horst. A Framework of Software Measurement. Walter de Gruyter, 2013.
- MCCABE, T. J. A Complexity Measure. IEEE Transactions on Software Engineering, 1976.
- Radon — Python code metrics. <https://radon.readthedocs.io>